

File permissions in Linux

Project description

The research team at my organization needs to update the file permissions for certain files and directories within the `projects` directory. The permissions do not currently reflect the level of authorization that should be given. Checking and updating these permissions will help keep their system secure. To complete this task, I performed the following tasks:

Check file and directory details

The following code shows how I used Linux commands to determine the existing permissions set for a specific directory in the file system.

```
researcher2@fc9f7486ec07:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Oct  1 09:21 .
drwxr-xr-x 3 researcher2 research_team 4096 Oct  1 11:09 ..
-rw--w---- 1 researcher2 research_team  46 Oct  1 09:21 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Oct  1 09:21 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Oct  1 09:21 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Oct  1 09:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_t.txt
```

Describe the permissions string

The 10-character string can be deconstructed to determine who is authorized to access the file and their specific permissions. The characters and what they represent are as follows:

- 1st character: This character is either a `d` or hyphen (`-`) and indicates the file type. If it's a `d`, it's a directory. If it's a hyphen (`-`), it's a regular file.
- 2nd-4th characters: These characters indicate the read (`r`), write (`w`), and execute (`x`) permissions for the user. When one of these characters is a hyphen (`-`) instead, it indicates that this permission is not granted to the user.
- 5th-7th characters: These characters indicate the read (`r`), write (`w`), and execute (`x`) permissions for the group. When one of these characters is a hyphen (`-`) instead, it indicates that this permission is not granted for the group.
- 8th-10th characters: These characters indicate the read (`r`), write (`w`), and execute (`x`) permissions for other. This owner type consists of all other users on the system apart from the user and the group. When one of these characters is a hyphen (`-`) instead, that indicates that this permission is not granted for other.

For example, the file permissions for `project_t.txt` are `-rw-rw-r--`. Since the first character is a hyphen (`-`), this indicates that `project_t.txt` is a file, not a directory. The second, fifth, and eighth characters are all `r`, which indicates that user, group, and other all have read permissions. The third and sixth characters are `w`, which indicates that only the user and

group have write permissions. No one has execute permissions for `project_t.txt`.

Change file permissions

The organization determined that other shouldn't have write access to any of their files and group shouldn't have read access to the file named `project_m.txt`. To comply with this, I referred to the file permissions that I previously returned.

```
researcher2@fc9f7486ec07:~/projects$ chmod o-w project_k.txt
researcher2@fc9f7486ec07:~/projects$ chmod g-r project_m.txt
researcher2@fc9f7486ec07:~/projects$ chmod u-w,g-w .project_x.txt
researcher2@fc9f7486ec07:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Oct  1 09:21 .
drwxr-xr-x 3 researcher2 research_team 4096 Oct  1 11:09 ..
-r----- 1 researcher2 research_team  46 Oct  1 09:21 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Oct  1 09:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_k.txt
-rw----- 1 researcher2 research_team  46 Oct  1 09:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_t.txt
```

Change file permissions on a hidden file

The research team at my organization recently archived `project_x.txt`. They do not want anyone to have write access to this project, but the user and group should have read access.

```
researcher2@fc9f7486ec07:~/projects$ chmod u-w,g-w .project_x.txt
researcher2@fc9f7486ec07:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Oct  1 09:21 .
drwxr-xr-x 3 researcher2 research_team 4096 Oct  1 11:09 ..
-r----- 1 researcher2 research_team  46 Oct  1 09:21 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Oct  1 09:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_k.txt
-rw----- 1 researcher2 research_team  46 Oct  1 09:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_t.txt
researcher2@fc9f7486ec07:~/projects$ chmod g+r .project_x.txt
```

Change directory permissions

My organization only wants the `researcher2` user to have access to the drafts directory and its contents. This means that no one other than `researcher2` should have execute permissions.

The following code demonstrates how I used Linux commands to change the permissions:

```
researcher2@fc9f7486ec07:~/projects$ chmod g-x drafts
researcher2@fc9f7486ec07:~/projects$ ls -l
total 20
drwx----- 2 researcher2 research_team 4096 Oct  1 09:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_k.txt
-rw----- 1 researcher2 research_team  46 Oct  1 09:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Oct  1 09:21 project_t.txt
researcher2@fc9f7486ec07:~/projects$
```

Summary:

I changed multiple permissions to match the level of authorization my organization wanted for files and directories in the `projects` directory. The first step in this was using `ls -la` to check the permissions for the directory. This informed my decisions in the following steps. I then used the `chmod` command multiple times to change the permissions on files and directories.